钟森翔

基本信息			
姓　　名	钟森翔	求职意向	Unity 客户端开发（实习）
手　　机	13336915533	电子邮箱	2318460152@qq.com
GitHub	https://github.com/AKI12345asa		
教育背景			
起止时间	2024 ~ 2028		
学　　校	杭州电子科技大学		
专　　业	计算机科学与技术		
学　　历	本科		
主修课程	数据结构与算法，软件工程,网页设计，线性代数		
英语水平	CET-6（英语六级）		
专业技能			
语言与引擎	C#,C++；Unity 2022 LTS，Addressables、Cinemachine、HybridCLR,Odin Inspector；熟悉 Unity 生命周期、协程与异步。		
架构能力	能独立设计并落地分层框架（框架 / 业务 / 配置三层），熟悉配置驱动、接口隔离、对象池、事件系统、状态机，以及游戏对象行为的优先级抢占式行为仲裁。		
性能与内存	熟悉 GC 来源与优化手段、托管与原生内存边界；熟练使用 `NativeArray` / `NativeStream` / JobSystem + Burst；有结构体 SoA 布局、避免每帧分配、原生容器生命周期与引用计数的实践经验。		
底层与算法	手写 GJK / EPA 凸体碰撞检测、Morton 码 LBVH、两层 BVH 空间加速；理解坐标变换、支撑函数与 Minkowski 差。		
工具链	能写编辑器扩展（技能时间轴编辑器、GOAP 编辑器、节点树编辑器），会使用 Odin 多态序列化与资产化配置开发工具		
工程素养	关注原生内存泄漏、配置健壮性、大文件拆分；有把设计沉淀成文档的习惯。		
项目经历			
项目名称	**MyFrame**	项目角色	独立开发
技术栈	C# · Unity 2022.3 · URP · Addressables　|　https://github.com/AKI12345asa/MyFrame		
主要工作	• 从零设计并实现 **10 个子系统**（动画、技能、战斗、GOAP、节点树、对象池、UI、事件、存档、时间），全部收敛在单一 asmdef 内，业务层不依赖任何第三方框架。 • **动画系统**：放弃 Animator Controller，改用 Playable 手动编排——转场逻辑写在脚本里便于调试；片段与混合数据独立成配置便于复用；提供统一动画事件与可跨节点类型混排的 `AnimationChain`。 • **技能系统**：把配置里的每个数据都视为“某一帧触发的事件”，用累计时间推进时间轴而非帧计数，一次 Update 可推进多个逻辑帧，使技能表现与帧率解耦。 • **GOAP**：实现 AND/OR 树——参考 A*使用启发函数取“估计最小花费前置条件满足者代价之和”，效果匹配采用乐观判定；在短规划生命周期、高决策频率的场景下优先保证可解释性与可视化调试。 • **节点树**：框架只提供链路与编辑器、不内置执行器，执行语义交由使用方实现（在 ARPG 项目中以 NPC 对话树落地验证）。		

项目名称	**ARPG 战斗原型**	项目角色	独立开发
技术栈	C# · Unity · MyFrame · Odin		
主要工作	• 在 MyFrame 之上搭建完整战斗原型：两套可插拔动作模块（自由移动 6 个状态、剑盾 10 个状态）、9 个技能行为、锁定相机、以及 GOAP 驱动的敌人 AI。 • **优先级抢占式行为仲裁**：用 `struct` 描述一次行为申请（动作 / 优先级 / 检查函数 / ID），handler 走对象池，使“切状态”这一高频操作几乎零分配；高优先级可打断低优先级，同时支持用检查函数让“条件满足”覆盖“优先级更高”。 • **状态与外部解耦**：状态自身持续持有行为槽位，外部可用不同 behaviour 借入；当被低优先级行为占用时，状态能凭自身占位重新夺回，避免优先级反转。 • **技能与输入解耦**：按键只负责 `ReleaseSkill(KeyData)`，具体释放哪个技能由配置决定，因此同一套输入代码可适配不同武器。 • **表现层注册制**：相机与打击检测以处理器形式在角色初始化时注册（玩家 6 个相机处理器 + 5 个检测处理器，敌人按需），同一份技能配置在不同角色上表现不同，无需修改技能数据。		
项目名称	**自定义碰撞检测系统**	项目角色	独立开发
技术栈	C# · JobSystem · Burst ｜ https://github.com/AKI12345asa/ColliderSystem		
主要工作	• 从零实现凸体碰撞系统，完整覆盖物理管线四段：**实体收集 → 粗筛 → 精算 → 求解与事件。** • **粗筛**：基于 Morton 码排序构建 LBVH（线性 BVH）网格三角形另外维护本地 BVH 并做引用计数缓存，与物体级顶层 BVH 组成两层结构。 • **精算**：手写 GJK 判交与 EPA 求穿透深度，按“点集 / 三角形 / 盒 / 球 / 胶囊”的组合提供多套支撑函数重载。 • **并行**：全流程跑在 JobSystem + Burst 上；用 `NativeStream` 收集并行结果避免写竞争，用 `IJobParallelForTransform` 并行回写变换。 • **事件与求解**：双缓冲 `HashSet` 对比前后两帧碰撞对，无分配地判定 Trigger 的 Enter / Stay / Exit；MTV 按逆质量加权分配，使重的一方位移更小。 • **内存治理**：把实体与包围盒数组改为容量池 + 逻辑计数，所有 Job 只接收 `GetSubArray(0, count)` 视图，杜绝“把容量当成物体数量”；补齐 `Dispose` 与运行时生命周期钩子，使 Persistent 原生内存在退出时零泄漏。		
项目名称	**ARPG 原型（早期版本）**	项目角色	独立开发
技术栈	C# · Unity		
主要工作	• 打通完整游戏闭环：主菜单 → 建角（外观装配）→ 场景异步加载 → 战斗 / 探索 → 对话接取任务 → 背包 / 装备 / 商店 → 存档 → 返回主菜单，流程可反复进入，场景之间用返回点与自动存档衔接。 • **配置驱动的内容扩展**：角色、敌人、物品、任务、场景统一抽象为配置资产，新增职业 / 敌人 / 物品 / 任务靠配表完成而不改代码；技能、GOAP、对话树、UI 窗口均由框架的特性 + 反射机制自动注册，业务层只负责填内容。 • **聚合式存档**：存档系统只负责序列化，各业务模块（玩家、任务、NPC、商店）在自己的回调里保存自己那一段，新增要存档的系统不需要改存档代码；自动存档挂在切场景钩子上，配合手动槽位与读档恢复。 • **任务与对话打通**：任务参数区分自动完成与交付完成，对话树里可直接挂接取 / 完成 / 条件判断节点；NPC 参数按场景隔离并持久化，同名 NPC 在不同场景互不影响。 • **操作与表现收口**：窗口按四层 Layout 管理遮罩与互斥，打开窗口自动锁角色操作与相机；交互选中用射线 + Bounds 求交 + 角度筛选，描边走 CommandBuffer 蒙版而非 URP Renderer Feature。 • 项目复盘后将可复用部分抽象为 MyFrame 并重写战斗层，即上述前两个项目。		